

Paillier-Based Aggregation Protocol

Phase 1: Setup (Trusted Third Party)

TP selects safe primes p, q ; sets $N = pq$. Picks $H \in \mathbb{Z}^*_{N^2}$. Publishes (N, H) .
Aggregator generates $sk_A \in \mathbb{Z}^*_{N^2}$. Each voter chooses $sk_i \in [0, N^2)$ independently.

Phase 2: Voter Encryption

$c_i = (1 + x_i \cdot N) \cdot H^{sk_i} \bmod N^2$
 $aux_i = pk_A^{sk_i} = H^{sk_A \cdot sk_i} \bmod N^2$
 x_i = packed votes (25-bit slots)
WASM module (Emscripten + libtommath)

Phase 3: Collector

$aux = \prod aux_i \bmod N^2$
 $= H^{sk_A \cdot \sum sk_i} \bmod N^2$
Sends aux to aggregator
Backend (Node.js + PostgreSQL)

Phase 4: Aggregation (libtommath native C module)

- Product: $\Pi = \prod c_i \bmod N^2 = (1 + (\sum x_i)N) \cdot H^{\sum sk_i} \bmod N^2$
- Exponentiate: $P = \Pi^{sk_A} \bmod N^2 = (1 + (\sum x_i)N)^{sk_A} \cdot H^{sk_A \cdot \sum sk_i} \bmod N^2$
- Divide mask: $P' = P \cdot aux^{-1} \bmod N^2 = (1 + (\sum x_i)N)^{sk_A} \bmod N^2$
- L function: $L(P') = (P' - 1) / N = sk_A \cdot (\sum x_i) \bmod N$
- Recover sum: $R = L(P') \cdot sk_A^{-1} \bmod N = \sum x_i \bmod N$
- Unpack: Extract 25-bit slots from $R \rightarrow$ vote counts per candidate

Phase 5: Result

$R = \sum x_i = v_1 \cdot 2^{25(k-1)} + v_2 \cdot 2^{25(k-2)} + \dots + v_k \cdot 2^0 \rightarrow$ Vote counts $[v_1, v_2, \dots, v_k]$